FIELD GUIDE AI COST ENGINEERING · OCT 2026

AI FinOps

See, cut and control your LLM bill **before it controls you.**

RECEIPT · ONE AGENT TASK
20 steps · illustrative mid-tier model

input tokens 730,000	$1.460
output tokens 12,000	$0.120
TOTAL	$1.580
+ prompt caching	$0.415
+ caching + batch	$0.208

-87%

98%
of FinOps teams now manage AI spend

3 in 4
can't prove AI outcomes to their CFO

Anatomy of a bill · 8 cost levers · Unit economics · Cheat sheet

THE PLAYBOOK · SWIPE →

Inform see every token › **Optimize** cut the waste › **Operate** govern the spend

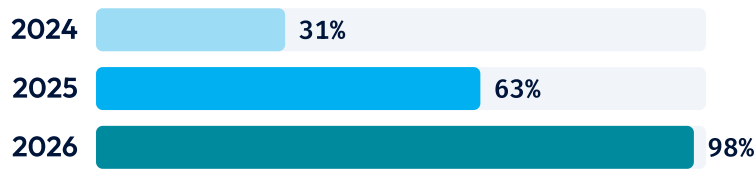
§01 THE CONCEPT

What Is AI FinOps, and Why Now?

In one line

Making every token, GPU hour and agent run **visible, owned and worth its cost**.

FINOPS TEAMS MANAGING AI SPEND



State of FinOps 2026, FinOps Foundation · 1,192 respondents, $83B+ cloud spend

WHY CLOUD FINOPS HABITS BREAK ON AI

Priced per token
Not per server hour. The cost moves with every prompt.

Output costs ~5× input
Long answers and reasoning are the pricey part.

Agents loop
Each step re-sends the whole context.

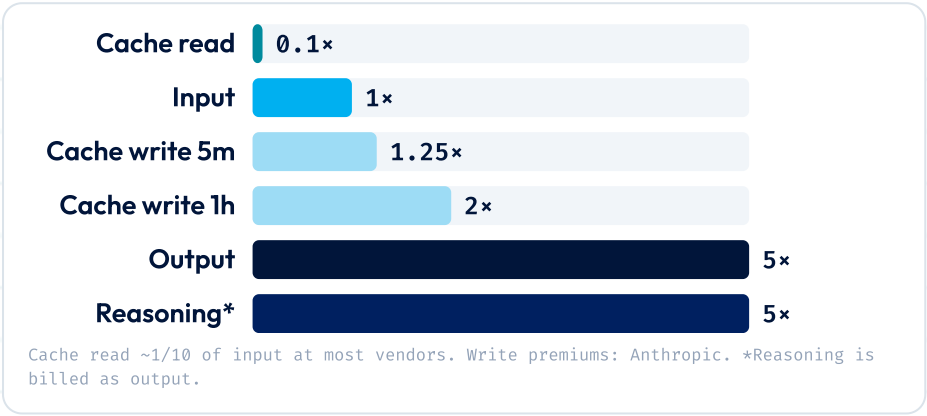
Users drive spend
One feature launch can multiply the bill.

#1 FinOps for AI is the top priority, and AI cost management the top missing skill.

§02 WHERE THE MONEY GOES

Anatomy of an AI Bill

Not all tokens cost the same. Typical price **relative to one input token**:



Cache read ~1/10 of input at most vendors. Write premiums: Anthropic. *Reasoning is billed as output.

THE LINES PEOPLE FORGET

GPUs
Self-hosted models, fine-tuning, idle reserved capacity.

Vector DB & storage
Embeddings, indexes, logs you keep forever.

Agent sandboxes
Code execution, browsers, tool calls.

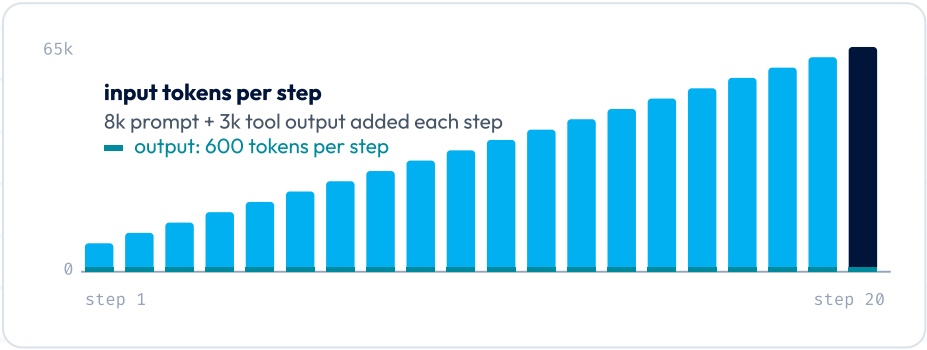
Gateway fees
e.g. OpenRouter adds 5.5% on card credits.

$ A token bill without a **token-type breakdown** is a guess.

§03 WHY AGENT BILLS EXPLODE

The Agent Loop Trap

Every step re-sends the system prompt, tools and all previous tool output.



730K
input tokens for one task

61 : 1
input to output ratio

92%
of the uncached bill is input

SO THE FIXES TARGET INPUT FIRST

- ✓ Cache the stable prefix (system prompt + tools)
- ✓ Trim or summarise old tool output (compaction)
- ✓ Cap steps per task, and alert on runaway loops

Worked example: 20 steps, 8,000-token prompt + tools, +3,000 tool tokens and 600 output tokens per step (ajayk.sh, Oct 2026).

Σ In agents, **input is the bill.**

AI Coach John FOLLOW | PROITBRIDGE Strive For Better Future REPOST

§04 THE FINOPS LIFECYCLE, APPLIED TO AI

Inform → Optimize → Operate

The FinOps Foundation's three phases. It's a loop, not a project.

01 **Inform** see it	**Who spent what, on which model, for which feature?**	Tag every call · Token-type breakdown · Showback dashboards · Cost per task
02 **Optimize** cut it	**Same quality, fewer dollars?**	Prompt caching · Model routing · Batch API · Output caps · Compaction
03 **Operate** govern it	**Who can spend how much, and what happens at the limit?**	Team budgets · Gateway limits · Anomaly alerts · Chargeback

ORDER MATTERS

Showback show teams their cost → **Trust the data** tags are complete → **Chargeback** teams pay their bill

→ Showback **before** chargeback. Bad data kills trust fast.

§05 INFORM · VISIBILITY & ALLOCATION

Tag Every LLM Call

Visibility is the #1 AI cost challenge. Allocating spend to business units is #2.

App adds tags → **Gateway** logs tokens + $ → **Cost table** FOCUS-style rows → **Showback** per team, feature

span attributes · OpenTelemetry GenAI

```
# standard (still "Development" status)
gen_ai.response.model                "model-name"
gen_ai.usage.input_tokens            8450
gen_ai.usage.output_tokens           612
gen_ai.usage.cache_read.input_tokens 8000
gen_ai.usage.reasoning.output_tokens 240
# your own business tags
app.team        "support"
app.feature     "ticket-summary"
app.customer    "acme-co"
```

Tag at the source
Missing tags can't be fixed later. Reject untagged keys.

One key per team
Easiest allocation when tools lack tag support.

∑ No tag, no owner. No owner, **no optimization**.

§06 INFORM · THE METRIC THAT MATTERS

Unit Economics: Cost per Outcome

Total spend tells you nothing. Divide it by what the business gets.

Cost per task (p95)

`spend / completed tasks`. Track p95 too, not just the average. Loops hide in the tail.

Cost per outcome

`spend / successful outcomes`. Failed and retried tasks count too.

EXAMPLE: AN AI SUPPORT AGENT

Price charged per resolved ticket

$0.99

Model cost per ticket (~3,700 tokens, small model)

`$0.0037 = 0.4% of the price`

Intercom Fin charges $0.99 per outcome. Model cost: Anthropic's support example (~$37 per 10,000 tickets on a Haiku-class model).

WHAT THIS TELLS YOU

- ✓ Model tokens are often a small slice of the unit cost
- ✓ Resolution rate moves margin more than model price
- ✓ Price AI features per outcome, budget them per task

÷ Report **$ per outcome** to the CFO, not $ per month.

§07 OPTIMIZE · LEVER 1

Prompt Caching: the First Win

The cache matches on an **exact prefix**. Put everything stable first.



~90%
off cached input at most vendors

1.25–2×
cache-write cost on Anthropic (5 min / 1 h)

6,144
min tokens for Gemini 4 Argon implicit cache

CACHE KILLERS

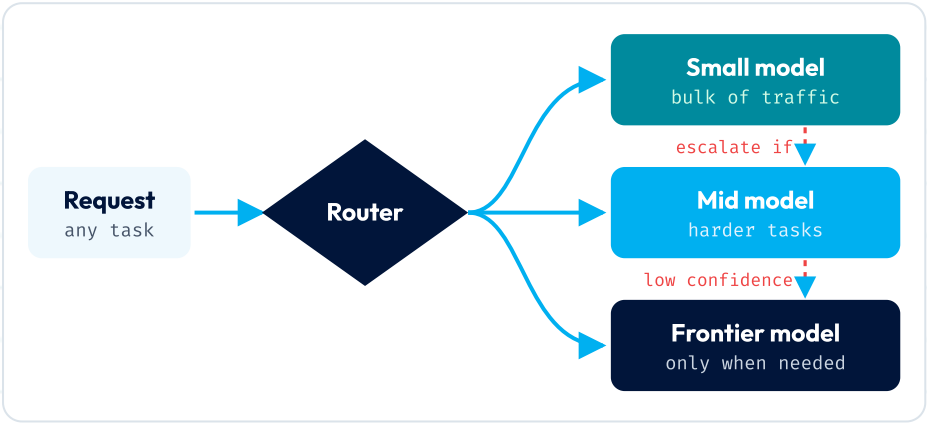
- ✕ Timestamps or user names at the top of the system prompt
- ✕ Re-ordering tools or examples between calls
- ✕ Paying write premiums on prompts that are rarely reused

% Track **cache hit rate** as a first-class metric.

§08 OPTIMIZE · LEVER 2

Model Routing by Task

Most requests don't need your most expensive model.



86%
of enterprises use or are evaluating model routers

4×
router users are this much likelier to show AI value

ROUTE SAFELY

- ✓ Build an **eval set** first. Route only where quality holds
- ✓ Watch the **escalation rate**. If it creeps up, retune

State of Tokenomics, Tokenomics Foundation, Sept 2026 (472 responses).

Route by **difficulty**, not by habit.

§09 OPTIMIZE · LEVERS 3–8

Six More Cost Levers

Batch API — ~50% off

Async jobs: evals, backfills, nightly summaries.

Cap output — pricey 5×

Set `max_tokens`. Ask for JSON, not prose.

Lower effort — fewer tokens

Use low thinking or effort levels for simple tasks.

Compaction — less input

Summarise old turns and tool output in long agents.

Tool cache — -35% calls

Reuse search and API results within a session.

Off-peak — up to 50%

Some vendors (e.g. DeepSeek) discount off-peak hours.

STACKED ON ONE AGENT TASK



Same 20-step task, illustrative mid-tier pricing. Tool-cache figure: AgentSysBench (Aug 2026).

Levers **stack**. Cache first, then batch, then route.

§10 OPERATE · GUARDRAILS

Budgets That Actually Stop Spend

A dashboard won't stop a runaway agent at 3 a.m. A hard limit at the gateway will.

LiteLLM proxy · per-team budget

```
curl -X POST "$PROXY/team/new" \
  -H "Authorization: Bearer $ADMIN_KEY" \
  -d '{
    "team_alias": "support-bot",
    "max_budget": 500,
    "budget_duration": "30d"
  }'
# over budget → requests rejected (422, or 429)
```

THE GUARDRAIL STACK

Hard limits
Per key or team at the gateway (LiteLLM, Cloudflare AI Gateway).

Alerts
50% / 80% / 100% of budget, plus daily spikes.

Forecast ranges
Spend swings with usage. Give a range, not a point.

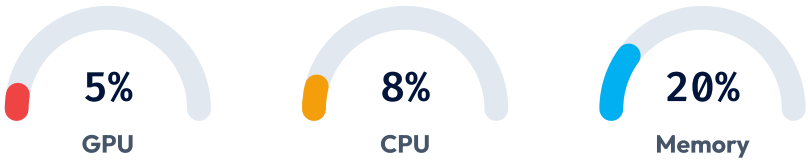
Loop caps
Max steps and max tokens per agent task.

≤ Every API key gets **a budget and an owner**.

§11 SELF-HOSTING & GPUS

The 5% GPU Problem

Average utilisation across Kubernetes clusters in one 2026 report:



Cast AI 2026 Kubernetes Optimization Report (one vendor's customers, GPU data to Apr 2026).

FIX THE GPU LINE

Allocate by usage

A $100 GPU used 50% charges $50 to that container (Kubecost).

Cost per 1M tokens

OpenCost exposes `llm_cost_per_million_tokens` per model.

Share GPUs

Pack small models onto one GPU instead of one each.

Scale to zero

Idle dev and eval endpoints shouldn't hold GPUs.

API? API vs self-host: compare **$ per 1M tokens at real utilisation**.

§12 READ BEFORE YOU FORECAST

Pricing Fine Print

Your price-per-token can jump without any code change.

GOTCHA	WHAT HAPPENS	IMPACT
Long-context tier	Price doubles above ~200k–272k input tokens on some vendors	2×
Intro pricing ends	e.g. Gemini 4 Argon: $2/$10 now, $4/$20 later	2×
New tokenizer	Same text, more tokens after a model upgrade	up to +30%
Regional inference	Pinning data to a region costs extra at US vendors	+10%
Reasoning tokens	Hidden thinking is billed as output	5× input
Cache storage	Some vendors bill cached tokens per hour stored	per token-hr
Off-peak windows	Discounts vanish in peak hours	±50%

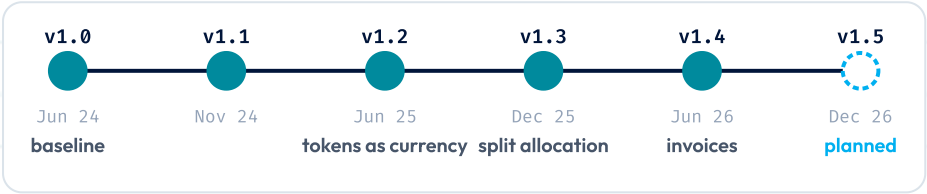
From an Oct 2026 survey of vendor pricing pages. Re-check pricing pages before you forecast.

! Pin versions. Re-run your cost model on **every model upgrade**.

§13 STANDARDS & TOOLING

The AI FinOps Stack

FOCUS = the FinOps Open Cost and Usage Specification, one billing format across providers.



The gap

No FOCUS version has a **model name or token count column** yet. Today you join billing data with gateway or OTel logs yourself.

TOOLS BY JOB

Gateways	LiteLLM, Portkey, Cloudflare AI Gateway, OpenRouter
Observability	Helicone, OpenTelemetry GenAI attributes
GPU / K8s cost	OpenCost, Kubecost
Cloud native	Bedrock inference profiles, Azure token meters
Billing data	FOCUS exports from AWS, Azure, GCP, Snowflake, Databricks

U Billing says **how much**. Gateway logs say **who and why**. Join them.

§14 THE REAL PROBLEM

The Value Gap

Cutting cost is the easy half. Proving AI is worth it is the hard half.

3 in 4
enterprises can't confidently prove AI outcomes to their CFO

23% vs 4%
want granular data from vendors, vs discounts

TOP AI COST CHALLENGES, IN ORDER

1. **Visibility**
   Every provider prices differently
2. **Allocation**
   Which business unit owns this spend?
3. **Value and ROI**
   Did this spend move a business metric?

CLOSE THE GAP

- ✓ Pair every AI feature with **one business metric** before launch
- ✓ Report cost per outcome next to that metric, monthly

State of FinOps 2026 (challenges) · State of Tokenomics, Sept 2026 (3 in 4, 23% vs 4%).

ROI Teams want **data**, not discounts.

§15 SAVE THIS

AI FinOps Cheat Sheet

SYMPTOM	LEVER	WATCH
"Who spent this?"	Tag every call, one key per team	`% tagged`
Agent bills explode	Cache prefix, compact, cap steps	`tokens / task`
Same prompt, full price	Static content first	`cache hit %`
Frontier model everywhere	Router + eval set	`escalation %`
Slow jobs at live prices	Batch API	`% batched`
Long, wordy answers	max_tokens, JSON output	`output / call`
Runaway spend	Gateway hard limits + alerts	`budget burn`
Idle GPUs	Usage allocation, scale to zero	`GPU util %`
Price jumps	Pin versions, re-run cost model	`$ / 1M tokens`
"Is it worth it?"	Cost per outcome vs metric	`$ / outcome`

Inform → Optimize → Operate. Report $ per outcome, not $ per month.

Sources: FinOps Foundation State of FinOps 2026 · Tokenomics Foundation State of Tokenomics (Sept 2026) · ajayk.sh FinOps for AI (Oct 2026) · SiliconANGLE FinOps X 2026 · Cast AI 2026 report · vendor pricing pages.

AI Coach John FOLLOW PROITBRIDGE Strive For Better Future REPOST